OOAD RBAC0 API 答辩手册

E 负责接口优先 其余接口逐项查阅 结合 31 题答辩追问

本手册面向月底验收，帮助我从老师点名的路由快速说出用途、输入、输出、所需权限、目标数据范围和失败处理。前四项是 E 负责的组织与导入接口，后面列出权限中心其余 41 项。最后收录上一版 Markdown 的 31 个源码追问与参考答案。

核对基线：本地仓库 b9ea4cf，OpenAPI 合约 rbac-api.yaml；接口行为参考 docs/04-api.md 与结构化后端。此 Word 是答辩速查副本，接口变更以仓库 OpenAPI 和源码为准。文中“失败处理”同时包含契约和实现中的业务边界；未逐项运行端到端实测，不把代码阅读当成实测结论。

权限中心共 45 项：GET 19、POST 17、PATCH 4、DELETE 5、PUT 0。模拟业务系统的接口另列，不计入这 45 项。

阅读和答题方法

先说业务场景，再说 HTTP 方法与路径、输入字段、输出 data，接着解释两道权限关口：入口操作权限由 OpenAPI 的 x-required-permission 与拦截器判定；目标对象能否访问，由 Service 按所属部门或公司再次校验。最后补充参数错误、越界、状态冲突及“200 但部分失败”的情况。

通用规则	答辩时的准确说法
基础地址与包装	基础路径 /api/v1；成功通常是 HTTP 200，响应含 code、message、data、timestamp。
入口鉴权	除登录外须 Bearer 令牌。@self 表示登录即可调用该类接口，不等于可操作任意人。
常见失败	400 参数；401 未认证；403 缺操作权限或超范围；404 对象不存在；409 重复或状态冲突。具体路由以源码为准。
重要例外	authz/check 的 allowed=false 是正常判定，HTTP 200；批量和导入的 200 也不保证全部成功，要查 data 内容。
PUT 与 PATCH	这 45 项没有 PUT。用户/角色 PATCH 只改非 null 字段；菜单 PATCH 按 SaveMenu 覆盖可编辑字段，是需说明的设计边界。

第一部分 E 组织与导入接口

01 GET /companies

用途 公司列表（只含管理范围涉及的公司）

输入 Bearer 令牌；无查询参数。

输出 CompanyView[]：id、companyCode、companyName、rootDepartmentId。

所需权限 rbac:org:view

目标数据范围 只列出持有 org:view 的范围涉及且已配置根部门的公司；超级管理员可见所有已配置根部门的公司。

失败处理 无入口权限 403；未登录或令牌失效 401。空结果是合法的 200。

契约细节 需要 rbac:org:view。只返回该权限管理范围涉及且已配置根部门的公司，超级管理员可查看所有已配置根部门的公司。 data 为包含 id、companyCode、companyName、rootDepartmentId 的数组，用于选择部门树和批量授权目标，不分页。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 923 行；按路径在对应 Controller、Service 中搜索。

02 GET /departments/tree

用途 部门树（管理范围内的部门及通往它们的祖先）

输入 必填 query：companyId（先从 /companies 取）。

输出 DeptNode[]：部门 id、parentId、名称、层级、负责人、children。

所需权限 rbac:org:view

目标数据范围 按 org:view 作用部门返回该部门及下级，并补上显示路径所需祖先；祖先可见不代表其旁支可管理。

失败处理 companyId 缺失或格式错 400；公司不存在 404；公司存在但无可见部门返回 200 和空数组。

契约细节 需要 rbac:org:view，companyId 必填，公司不存在时返回 404。 返回管理范围内部门及下级，并补充通往这些部门的祖先节点以便展示；祖先可见不表示可管理祖先下的其他部门。 data 为包含层级、负责人及 children 的部门树；公司没有可见部门时返回空数组。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 935 行；按路径在对应 Controller、Service 中搜索。

03 POST /imports/organization

用途 导入部门信息表与员工信息表

输入 query：companyCode、companyName；multipart：departments 和 employees 两份 xlsx，读各自首个工作表。

输出 ImportReport：counts、notes、anomalies；200 不代表每一行都导入成功。

所需权限 rbac:org:import

目标数据范围 入口权限为 org:import；导入写入目标公司、部门与员工；不会自动分配角色或因负责人身份授予管理权。

失败处理 缺参数/无法读取文件可返回 400；无法唯一定位的行记入 anomalies，有效行可提交；数据库异常则由事务回滚。

契约细节 需要 rbac:org:import。通过 query 指定公司编码与名称，multipart/form-data 同时上传 departments、employees 两份 xlsx。 按项目部门信息表、员工信息表格式读取首个工作表，目标公司不存在时创建，已存在的部门和用户名跳过。 无法唯一定位部门、负责人等数据时记录 anomalies，并跳过无法处理的数据；有效数据仍可导入。 不自动分配角色，部门负责人也不会因此获得管理权限。200 返回 counts、notes、anomalies，应检查异常后人工授权。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 950 行；按路径在对应 Controller、Service 中搜索。

04 POST /imports/permissions

用途 导入应用软件功能权限清单（业务系统、模块、资源、权限、五类角色模板）

输入 multipart：file，一份项目格式的功能权限清单 xlsx。

输出 ImportReport：计数、说明、异常清单，含新增和回收统计。

所需权限 rbac:permission:create

目标数据范围 需 permission:create；修改业务系统、模块、资源、权限和五类模板角色的业务授权，不给用户直接指派角色；rbac:* 平台授权不由清单回收。

失败处理 文件无法读取/参数错误可返回 400；异常格保留原授权；功能点无法定位时暂停删除同步；200 仍须检查 anomalies。

契约细节 需要 rbac:permission:create，通过 multipart/form-data 的 file 上传项目格式的功能权限清单 xlsx，读取首个工作表。 建立或沿用系统、模块、资源及五类业务操作权限，并按清单同步五类角色模板的业务权限和本人数据限制。重复导入可新增、更新，也会回收清单中移除的模板业务权限；权限中心自身 rbac:* 权限保留。 异常单元格对应的原有授权保留；存在无法定位的功能点时暂停本次删除同步。导入不会给用户分配角色。 200 返回 counts、notes、anomalies，应检查异常清单及回收统计，变更授权后更新缓存版本。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 977 行；按路径在对应 Controller、Service 中搜索。

第二部分 其他 41 个权限中心接口

认证与鉴权

05 POST /auth/login

用途 登录

输入 JSON：username、password；不需要 Bearer 令牌。

输出 token、expiresIn、userId、username、realName、mustChangePwd。

所需权限 免登录

目标数据范围 按提交的用户名认证一个账号，不提供跨用户查询。

失败处理 用户名或口令错 40102；连续失败锁定 40103；口令正确但账号停用 40104；失败会计数并审计。

契约细节 使用用户名和口令登录，无需 Bearer 认证。成功返回 JWT、有效秒数、用户基本信息和 mustChangePwd。用户不存在、已删除或口令错误时返回 40102「用户名或口令错误」；连续口令错误 5 次后锁定 15 分钟， 锁定期间返回 40103。口令正确但账号已停用时返回 40104。 mustChangePwd=true 时须先修改本人口令，再重新登录。登录成功与失败均记录 LOGIN 审计事件。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 66 行；按路径在对应 Controller、Service 中搜索。

06 POST /auth/logout

用途 登出

输入 Bearer 令牌；无业务请求体。

输出 统一响应 data=null。

所需权限 @self

目标数据范围 只注销当前携带的令牌，编号存 Redis 至原到期时间。

失败处理 未认证 401；注销后的令牌再次调用会被拒绝。

契约细节 注销当前请求携带的 JWT，无需额外操作权限。令牌编号登记到 Redis，直至原到期时间前均被拒绝。 仅注销当前令牌；成功返回 data=null，并记录 LOGOUT 审计事件。已注销令牌再次调用时返回 401。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 99 行；按路径在对应 Controller、Service 中搜索。

07 GET /auth/me

用途 当前用户的基本信息、角色与有效权限

输入 Bearer 令牌；无参数。

输出 UserDetail：本人资料、角色指派、superAdmin、有效权限。

所需权限 @self

目标数据范围 固定查询当前登录者；首次登录须改密时仍可调用。

失败处理 无效、过期、已注销或账号已停用等令牌返回 401。

契约细节 根据 Bearer JWT 确定当前用户，无需传入用户 ID 或额外操作权限。 data 包含 user、角色指派 roles、超级管理员标识 superAdmin 和带来源角色、适用部门的 permissions。 首次登录尚未修改口令时也可调用，前端可据此提示改密并控制功能入口。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 112 行；按路径在对应 Controller、Service 中搜索。

08 POST /auth/password

用途 修改本人口令

输入 JSON：旧口令、新口令。

输出 统一响应 data=null；成功后需重新登录。

所需权限 @self

目标数据范围 只修改当前登录者口令，并使旧令牌版本失效。

失败处理 旧口令不对或新口令不符复杂度/重复旧口令返回 400；无效令牌 401。

契约细节 为当前登录用户修改口令，首次登录尚未改密时也可调用。必须提供正确旧口令；新口令至少 8 位， 同时包含大写字母、小写字母、数字和特殊字符，且不能与旧口令相同，否则返回 400。 成功后清除 mustChangePwd，当前及此前签发的全部令牌失效，须使用新口令重新登录。 成功返回 data=null，并记录 PASSWORD_CHANGE 审计事件。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 134 行；按路径在对应 Controller、Service 中搜索。

09 POST /authz/check

用途 鉴权：某用户能否对某部门的数据执行某操作

输入 JSON：userId 或 username（都不填查本人）； permission 或 resource+action；可选 targetDeptId。

输出 Decision：allowed、reason、matched、selfOnly、cacheHit 等；拒绝判定仍是 HTTP 200。

所需权限 @self

目标数据范围 查他人须在被查人的所属部门持有 authz:invoke；targetDeptId 是待判定记录部门，不可用于绕过查他人限制。

失败处理 请求字段不合法 400；调用者无权查他人 403；被判定者无授权通常返回 200 且 allowed=false。

契约细节 主体通过 userId 或 username 指定，都不填则查询调用者本人；同时填写时优先使用 userId。 权限优先取 permission，否则由 resource + action 拼接。targetDeptId 表示被操作数据所属部门， 不填时只判定是否具有该操作权限，不限定具体部门。 查询本人无需额外权限；查询他人时，调用者须在被查用户所属部门持有 rbac:authz:invoke，否则返回 403。 判定完成始终返回 200，是否放行见 data.allowed；拒绝原因见 reason。selfOnly=true 时， 业务系统还须按记录归属过滤本人数据。每次完成的判定记录 AUTH_SUCCESS 或 AUTH_FAILED。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 160 行；按路径在对应 Controller、Service 中搜索。

10 POST /authz/explain

用途 权限解释：与 check 相同的判定，另返回逐步的判定依据（trace）

输入 与 /authz/check 相同的 JSON。

输出 Decision 另带 trace 逐步说明判定依据。

所需权限 @self

目标数据范围 查他人须在被查人的所属部门持有 authz:explain。

失败处理 无权查他人 403；正常拒绝判定仍为 200、allowed=false。

契约细节 请求参数与 /authz/check 相同，使用同一套权限判定过程，额外通过 data.trace 返回逐步判定依据。 查询本人无需额外权限；查询他人时，须在被查用户所属部门持有 rbac:authz:explain，否则返回 403。 正常完成的拒绝仍返回 200 + allowed=false；matched 表示放行所依据的授权，超级管理员旁路时为空。 每次完成的判定记录 AUTH_SUCCESS 或 AUTH_FAILED。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 190 行；按路径在对应 Controller、Service 中搜索。

用户与角色指派

11 GET /users

用途 用户列表与搜索（只返回管理范围内的用户）

输入 query：keyword、departmentId、includeDescendants、status、userType、page、size。

输出 Page<User>：items、total、page、size。

所需权限 rbac:user:view

目标数据范围 查询条件与操作者 user:view 的部门管理范围取交集；默认排除 DELETED。

失败处理 参数不合法 400；缺 user:view 403；无匹配记录是空页。

契约细节 需要 rbac:user:view。按用户名、姓名或工号模糊搜索，可结合部门、用户状态和用户类型筛选。departmentId 配合 includeDescendants 决定是否查询下级部门，结果始终与调用者的管理范围取交集。 不传 status 时排除已删除用户；显式传 DELETED 可查询管理范围内的已删除用户。 data 为分页结果，按用户 ID 升序，page 默认 0、size 默认 20，实际每页限制为 1～200。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 219 行；按路径在对应 Controller、Service 中搜索。

12 POST /users

用途 新增用户（正式员工、临时用户或业务系统账号）

输入 JSON CreateUser：username、realName、departmentId、userType、employeeNo、phone、position、password。

输出 新建 User 基本信息；不附带任何角色。

所需权限 rbac:user:create

目标数据范围 目标部门必须在调用者 user:create 的可管理范围；公司由部门确定。

失败处理 用户类型字段组合非法 400；目标越界 403；用户名/工号重复或内部 ID 耗尽 409。

契约细节 需要在目标部门持有 rbac:user:create。公司由目标部门确定，userType 默认 EMPLOYEE，也支持 TEMPORARY、SYSTEM。 正式员工必须填写工号；临时用户不能填写工号、电话，用完由管理员停用（账号没有到期时间）。 用户名和非空工号必须唯一，已删除用户的用户名也不能复用，否则返回 409。 password 为空时采用配置的初始口令并要求首次登录修改；自定义口令须符合复杂度要求。 SYSTEM 为内部账号，ID 使用 1～9999，编号用完返回 409；员工与临时用户从 10000 起编号。 成功返回新用户基本信息，并记录 USER_CREATE；创建账号后仍需单独指派角色。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 251 行；按路径在对应 Controller、Service 中搜索。

13 POST /users/batch-status

用途 批量启用 / 停用 / 删除

输入 JSON：userIds、status、reason；status 可为 ACTIVE、DISABLED、DELETED。

输出 BatchResult：succeeded、failed、failures 等；200 可为部分成功。

所需权限 rbac:user:edit

目标数据范围 逐人按所属部门校验；删除还需 user:delete；本人和超级管理员受额外保护。

失败处理 空列表 400；失败用户记入 failures；不要以 HTTP 200 推断全部成功。

契约细节 接口入口需要 rbac:user:edit；逐人处理时，启用、停用校验 rbac:user:edit，删除校验 rbac:user:delete， 均要求权限覆盖该用户所属部门。userIds 不能为空，重复 ID 去重后处理。 固定采用 PARTIAL 模式：通过的执行，失败的在 failures 中返回用户 ID 和原因；不能变更本人或已删除账号， 普通管理员不能变更超级管理员账号。正常处理返回 200，不表示所有用户均成功，应检查 succeeded、failed 和 failures。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 275 行；按路径在对应 Controller、Service 中搜索。

14 GET /users/{id}

用途 用户详情：基本信息、角色、有效权限

输入 路径 id。

输出 UserDetail：用户信息、角色指派和有效权限。

所需权限 @self

目标数据范围 本人可查；查他人须在其所属部门有 user:view。

失败处理 无权查他人 403；用户不存在 404。

契约细节 本人可直接查询自己；查询其他用户须在其所属部门持有 rbac:user:view，超出管理范围返回 403。 data 包含用户基本信息、角色指派、超级管理员标识，以及带来源角色和适用部门的有效权限。 角色指派与有效权限分开返回，停用角色的指派仍可见，但不贡献有效权限；用户不存在时返回 404。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 302 行；按路径在对应 Controller、Service 中搜索。

15 PATCH /users/{id}

用途 修改用户（姓名、部门、电话、岗位）

输入 路径 id；JSON UpdateUser：realName、departmentId、phone、position；未传字段保持原值。

输出 更新后的 User 基本信息。

所需权限 rbac:user:edit

目标数据范围 须管理该用户当前部门；调动部门还要求新部门同公司且在 user:edit 范围。

失败处理 空白姓名、临时用户非空电话等 400；目标越界 403；用户/部门不存在 404。

契约细节 需要在用户当前所属部门持有 rbac:user:edit；普通管理员不能修改其他超级管理员账号，已删除用户不能修改。仅更新非 null 字段；姓名不能是空白，电话和岗位可传空字符串清空，临时用户不能设置非空电话。 调整部门时，调入部门须属于同一公司且也在管理范围内。成功返回更新后的用户基本信息，并记录 USER_UPDATE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 323 行；按路径在对应 Controller、Service 中搜索。

16 DELETE /users/{id}

用途 删除用户（逻辑删除，用户名不可复用，指派保留供审计）

输入 路径 id；可选 query reason。

输出 data=null；逻辑删除，保留审计所需指派。

所需权限 rbac:user:delete

目标数据范围 须在目标所属部门持有 user:delete；普通管理员不能删除超级管理员。

失败处理 越界 403；目标不存在 404；删除本人或已删除用户等状态冲突 409。

契约细节 需要在目标用户所属部门持有 rbac:user:delete。将状态改为 DELETED 并记录删除时间，保留角色指派供审计，用户名不可复用，既有令牌在下一次请求失效。不能删除本人或再次删除已删除账号，状态冲突返回 409； 普通管理员不能删除超级管理员账号。reason 可记录删除原因，成功返回 data=null，并记录 USER_DELETE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 341 行；按路径在对应 Controller、Service 中搜索。

17 PATCH /users/{id}/status

用途 启用或停用

输入 路径 id；JSON：status（ACTIVE 或 DISABLED）、reason。

输出 更新后的 User 基本信息。

所需权限 rbac:user:edit

目标数据范围 须在目标所属部门持有 user:edit；本人和超级管理员受保护。

失败处理 status 非法 400；目标越界 403；不能停用本人/恢复已删除用户等 409。

契约细节 需要在目标用户所属部门持有 rbac:user:edit，status 只接受 ACTIVE 或 DISABLED，删除应使用 DELETE 接口。 不能变更本人的状态，也不能恢复已删除账号；普通管理员不能变更超级管理员账号。 停用后既有令牌在下一次请求失效；角色指派保留。成功返回用户基本信息，并记录 USER_STATUS 及 reason。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 360 行；按路径在对应 Controller、Service 中搜索。

18 GET /users/{id}/roles

用途 用户的角色指派

输入 路径 id。

输出 Assignment[]：每条角色指派含 assignmentId、角色、适用部门等。

所需权限 @self

目标数据范围 本人可查；查他人须在其所属部门有 user:view。

失败处理 目标不存在 404；查他人无权 403。

契约细节 本人可查看自己的指派；查询他人须在其所属部门持有 rbac:user:view。 data 为指派数组，每项包含 assignmentId、角色状态、适用部门、授予人、授予时间和原因。 同一角色可在不同适用部门分别指派，停用角色的指派仍会返回；回收时使用 assignmentId。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 387 行；按路径在对应 Controller、Service 中搜索。

19 POST /users/{id}/roles

用途 为用户分配角色（ROLE_ASSIGN）

输入 路径用户 id；JSON：roleId、scopeDepartmentId（可省略）、reason。

输出 新建 Assignment 指派记录。

所需权限 rbac:assignment:assign

目标数据范围 用户所属部门和角色适用部门都须在操作者 assignment:assign 范围；且不可授出自己在适用部门没有的权限。

失败处理 越界/越权 403；用户或角色不存在 404；重复指派 409。

契约细节 需要在用户所属部门和适用部门均持有 rbac:assignment:assign，scopeDepartmentId 默认取用户所属部门，覆盖其下级。 用户须未删除、角色须启用，公司内角色只能分配到本公司的部门；适用部门可以在用户所属公司以外（跨公司授权），但用户所属部门与适用部门都须在调用者的管理范围内。 超级管理员角色只有超级管理员能分配；角色中各权限不能超出调用者在适用部门所持有的权限及本人数据限制。 相同用户、角色、适用部门的指派重复时返回 409。成功返回指派详情，更新授权缓存版本并记录 ROLE_ASSIGN。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 406 行；按路径在对应 Controller、Service 中搜索。

20 DELETE /users/{id}/roles/{assignmentId}

用途 回收一次角色指派（ROLE_REVOKE）

输入 路径用户 id 与 assignmentId；可选 query reason。

输出 data=null；只删这一条指派。

所需权限 rbac:assignment:revoke

目标数据范围 查适用部门的回收范围；回收本人指派不受管理范围限制，但入口仍需 assignment:revoke。

失败处理 指派不属此用户或不存在 404；越界 403；最后有效超级管理员不得回收，409。

契约细节 接口需要 rbac:assignment:revoke；回收他人指派还要求该权限覆盖指派的适用部门，回收本人指派不受管理范围限制。 assignmentId 必须属于路径中的用户， 不存在或不匹配时返回 404。只回收该条指派，被回收者过去授给其他人的指派保留。 他人的超级管理员角色只有超级管理员能回收；不能回收最后一个有效的超级管理员，否则返回 409 并回滚。 成功返回 data=null，更新授权缓存版本并记录 ROLE_REVOKE，reason 用于审计。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 444 行；按路径在对应 Controller、Service 中搜索。

21 GET /users/{id}/permissions

用途 用户的有效权限（每条带来源角色与适用部门）

输入 路径 id。

输出 EffectivePermission[]：权限码、来源角色、适用部门、selfOnly。

所需权限 @self

目标数据范围 本人可查；查他人须在其所属部门有 user:view。

失败处理 目标不存在 404；查他人无权 403。

契约细节 本人可查询自己；查询他人须在其所属部门持有 rbac:user:view。 data 为有效权限数组，包含权限码、来源角色、适用部门和 selfOnly。同一权限可因角色或适用部门不同出现多次， 按权限码、适用部门 ID 排序。超级管理员的旁路能力不展开为全量权限码，身份标识应查询用户详情的 superAdmin。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 466 行；按路径在对应 Controller、Service 中搜索。

角色与角色权限

22 GET /roles

用途 角色列表：名称、编码、用户数、权限数、状态

输入 query：keyword、status。

输出 Role[]：角色编码、名称、状态、用户数、权限数。

所需权限 rbac:role:view

目标数据范围 超级管理员看全部；其他人看全局角色及其管理范围涉及公司的角色。

失败处理 缺 role:view 403；无匹配是空数组。

契约细节 需要 rbac:role:view。返回全局角色及调用者管理范围涉及公司的角色，超级管理员可查看全部。 keyword 匹配角色编码或名称，status 可筛选 ENABLED、DISABLED；结果为按角色 ID 升序的数组，不分页。 每项包含角色基本信息、用户数和权限数；列表中的用户数按不同用户去重计算。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 488 行；按路径在对应 Controller、Service 中搜索。

23 POST /roles

用途 新增角色

输入 JSON CreateRole：roleCode、roleName、description、companyId。

输出 新角色基本信息；初始无权限和用户。

所需权限 rbac:role:create

目标数据范围 全局角色只能超级管理员创建；公司角色要求在公司根部门持有 role:create。

失败处理 编码/名称非法 400；越界 403；角色编码重复 409。

契约细节 需要 rbac:role:create。companyId 为空表示创建全局角色，仅超级管理员可操作； 创建公司内角色时，需要在该公司根部门持有此权限。 roleCode 必须以大写字母开头，由大写字母、数字和下划线组成，总长 2～64 位，且全局唯一；roleName 必填。 成功返回角色基本信息并记录 ROLE_CREATE，新角色尚无权限和用户指派，须分别授权与分配。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 510 行；按路径在对应 Controller、Service 中搜索。

24 GET /roles/{id}

用途 角色详情：基本信息与权限

输入 路径角色 id。

输出 RoleDetail：角色信息与授权权限。

所需权限 rbac:role:view

目标数据范围 公司角色受公司范围限制；全局角色可见。

失败处理 角色不存在 404；公司范围不可见 403。

契约细节 需要 rbac:role:view。全局角色可查看；公司内角色仅向管理范围涉及该公司的调用者或超级管理员开放。 data 包含 role 和 permissions，权限项列出资源、操作名称、selfOnly 和授予时间。 用户指派另通过 /roles/{id}/users 分页获取；角色不存在时返回 404，超出可见公司范围时返回 403。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 540 行；按路径在对应 Controller、Service 中搜索。

25 PATCH /roles/{id}

用途 修改角色名称、描述、启用停用

输入 路径 id；JSON UpdateRole：roleName、description、status；未传字段保留。

输出 更新后角色基本信息。

所需权限 rbac:role:edit

目标数据范围 全局角色仅超级管理员可改；公司角色要求在根部门持有 role:edit。

失败处理 状态非法 400；越界 403；超级管理员角色不能停用，409。

契约细节 需要 rbac:role:edit。全局角色仅超级管理员可修改；公司内角色要求在公司根部门持有该权限。 可修改名称、描述及 ENABLED、DISABLED 状态，不修改角色编码或所属公司；省略字段保留原值。 状态变化会更新授权缓存版本，停用后所有持有者在下一次请求失去该角色贡献的权限。 超级管理员角色不能停用。成功返回角色基本信息，并记录 ROLE_UPDATE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 560 行；按路径在对应 Controller、Service 中搜索。

26 DELETE /roles/{id}

用途 删除角色（内置角色不能删；仍有人持有时 409）

输入 路径角色 id。

输出 data=null；删除角色定义。

所需权限 rbac:role:delete

目标数据范围 全局角色仅超级管理员可删；公司角色按根部门 role:delete 校验。

失败处理 不存在 404；内置角色或仍有人持有时 409；越界 403。

契约细节 需要 rbac:role:delete。全局角色仅超级管理员可删除；公司内角色要求在公司根部门持有该权限。 内置角色或仍存在任何用户指派的角色不能删除，返回 409，应先人工回收指派。 成功删除角色及其角色权限关联，返回 data=null，并记录 ROLE_DELETE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 583 行；按路径在对应 Controller、Service 中搜索。

27 GET /roles/{id}/users

用途 角色详情「用户」：持有该角色的指派

输入 路径角色 id；query page、size。

输出 Page<Assignment>；total 按可见指派条数计。

所需权限 rbac:role:view

目标数据范围 先校验角色公司可见，再只返回适用部门在管理范围内的指派。

失败处理 角色不存在 404；无 role:view 或范围不足 403。

契约细节 需要 rbac:role:view，并先校验角色的公司可见范围。仅返回适用部门在调用者管理范围内的角色指派， 超级管理员可查看全部。data 为分页指派结果，page 默认 0、size 默认 20，实际每页限制为 1～200。 total 统计可见指派条数，同一用户在多个适用部门持有该角色时会有多条记录。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 599 行；按路径在对应 Controller、Service 中搜索。

28 POST /roles/{id}/permissions

用途 为角色授予权限（PERMISSION_ASSIGN）

输入 路径角色 id；JSON：permissionIds 数组。该接口请求不接收 selfOnly。

输出 更新后的 RoleDetail。

所需权限 rbac:grant:assign

目标数据范围 全局角色仅超级管理员；公司角色须在根部门有 grant:assign，且不得授出自己缺少的权限。

失败处理 列表为空 400；权限不存在 404；越权 403；超级管理员角色不接受显式授权。

契约细节 需要 rbac:grant:assign。全局角色仅超级管理员可修改；公司内角色要求在公司根部门持有该权限。 permissionIds 不能为空，重复 ID 去重，角色已持有的权限跳过；任何权限 ID 不存在时返回 404。 非超级管理员须在公司根部门持有每个待授予权限，且不能仅有 selfOnly 的本人范围，否则返回 403。 超级管理员角色采用旁路判定，不接受显式授权。成功返回角色详情，新增授权会更新缓存版本并记录 PERMISSION_ASSIGN。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 629 行；按路径在对应 Controller、Service 中搜索。

29 DELETE /roles/{id}/permissions/{permissionId}

用途 回收角色的一个权限（PERMISSION_REVOKE）

输入 路径角色 id 与权限 id。

输出 data=null；删除角色和权限的关联并更新授权版本。

所需权限 rbac:grant:revoke

目标数据范围 全局角色仅超级管理员；公司角色按根部门 grant:revoke 校验。

失败处理 角色/权限关联不存在 404；操作范围不足 403。

契约细节 需要 rbac:grant:revoke。全局角色仅超级管理员可修改；公司内角色要求在公司根部门持有该权限。permissionId 指向权限定义，必须已授予该角色，否则返回 404。 成功仅删除该角色与权限的关联，返回 data=null，更新缓存版本并记录 PERMISSION_REVOKE； 持有该角色的全部用户在下一次请求失去此角色贡献的该权限。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 664 行；按路径在对应 Controller、Service 中搜索。

批量指派与回收

30 POST /assignments/batch

用途 按分公司 / 部门 / 员工批量分配角色

输入 JSON：targetType、targetIds、includeDescendants、roleId、scopeDepartmentId、reason、mode。

输出 BatchResult：mode、committed、requested、succeeded、duplicated、failed、failures。

所需权限 rbac:assignment:assign

目标数据范围 按公司/部门/用户展开目标；每人校验所属部门、适用部门和授权上界。

失败处理 默认 ATOMIC 有一人失败则不写；PARTIAL 允许逐人失败；HTTP 200 须看 committed/failures。

契约细节 需要 rbac:assignment:assign。按 targetType 将公司、部门或用户 ID 展开为用户，重叠范围中的用户去重；部门目标默认包含下级。逐人校验用户部门、适用部门的管理权限、公司内角色的公司归属及禁止越权授予规则；适用部门可以在用户所属公司以外。 scopeDepartmentId 不填时，各用户以其所属部门为适用部门。 默认 ATOMIC：任何一人未通过校验则整批不写，返回 committed=false；PARTIAL 则写入通过的用户。 已持有同角色、同适用部门指派的人计入 duplicated，不算失败。200 表示处理完成，须检查 committed 和 failures。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 679 行；按路径在对应 Controller、Service 中搜索。

31 POST /assignments/batch-revoke

用途 按分公司 / 部门 / 员工批量回收角色

输入 与批量分配共用参数；scopeDepartmentId 为空时回收该角色的全部适用部门指派。

输出 BatchResult；200 后仍须查 committed、failed、failures。

所需权限 rbac:assignment:revoke

目标数据范围 按目标展开逐人检查适用部门回收范围；回收本人指派可例外。

失败处理 ATOMIC 整批失败可不写；PARTIAL 逐人报告；最后有效超级管理员回收返回 409。

契约细节 需要 rbac:assignment:revoke，按公司、部门或用户展开目标，逐人校验回收权限（覆盖各指派的适用部门）；回收本人指派不受管理范围限制。 scopeDepartmentId 为空时回收目标用户持有该角色的全部指派，否则只回收该适用部门的指派。 默认 ATOMIC，任一用户校验失败整批不执行；PARTIAL 执行通过者。无匹配指派的用户计入 duplicated，不算失败。 succeeded 按成功处理的用户数统计；回收最后一个有效超级管理员返回 409 并回滚。 被回收者过去授给他人的指派保留；200 后仍须检查 committed、failed 和 failures。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 699 行；按路径在对应 Controller、Service 中搜索。

资源与权限目录

32 GET /systems

用途 业务系统与功能模块（权限域）

输入 无业务参数。

输出 SystemView[]：系统与 modules 子数组。

所需权限 rbac:permission:view

目标数据范围 需要 permission:view；当前接口未按公司/部门再裁剪目录数据。

失败处理 入口未认证 401 或无 permission:view 403。

契约细节 需要 rbac:permission:view。data 为业务系统数组，每项包含系统 ID、编码、名称及所属 modules 数组，模块含 ID、编码和名称。按配置顺序返回，用于资源筛选及新建资源时选择 moduleId，不分页。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 720 行；按路径在对应 Controller、Service 中搜索。

33 GET /resources

用途 资源列表（BUSINESS 业务功能点 / MENU 菜单 / API 接口能力）

输入 query：systemId、type、keyword、page、size；type 可按 BUSINESS、MENU、API 筛选。

输出 Page<ResourceView>：资源与所属模块/系统名称。

所需权限 rbac:permission:view

目标数据范围 需要 permission:view；按查询条件筛选目录资源，未见组织作用域筛选。

失败处理 查询参数不合法 400；无 permission:view 403。

契约细节 需要 rbac:permission:view。可按业务系统、资源类型以及资源编码或名称关键字筛选。 data 为分页结果，items 包含资源基本信息、所属模块名和系统名。 page 默认 0、size 默认 50，实际每页限制为 1～500；资源类型包括 BUSINESS、MENU、API。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 732 行；按路径在对应 Controller、Service 中搜索。

34 POST /resources

用途 新增资源，并为给出的操作建立权限

输入 JSON CreateResource：moduleId、resourceCode、resourceName、resourceType、description、actionCodes。

输出 新建 ResourceView；同时可生成相关 Permission。

所需权限 rbac:permission:create

目标数据范围 需要 permission:create；写入指定系统模块的资源目录；实现未按公司/部门裁剪。

失败处理 模块/操作不存在可报 404；编码不合法 400；编码重复 409。

契约细节 需要 rbac:permission:create。moduleId 必须存在；resourceCode 使用小写字母、数字、连字符及冒号分段，且必须以所属业务系统编码加冒号开头。resourceType 默认 BUSINESS，资源编码重复返回 409。 actionCodes 中的每项须已存在于操作词汇表，按「资源编码:操作编码」同时创建权限；省略则只创建资源。 整个创建过程在同一事务中完成，成功返回资源基本信息，并记录 RESOURCE_CREATE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 749 行；按路径在对应 Controller、Service 中搜索。

35 GET /actions

用途 操作词汇表

输入 无业务参数。

输出 ActionView[]：id、actionCode、actionName。

所需权限 rbac:permission:view

目标数据范围 需要 permission:view；返回全局操作词汇表。

失败处理 入口未认证 401 或无 permission:view 403。

契约细节 需要 rbac:permission:view。data 为按配置顺序排列的操作数组，每项包含 id、actionCode、actionName。创建资源或权限时应使用此处已有的 actionCode，例如 view、create、edit、approve、export；本接口不分页。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 778 行；按路径在对应 Controller、Service 中搜索。

36 GET /permissions

用途 权限列表（Permission = Resource × Action）

输入 query：systemId、resourceId、keyword、page、size。

输出 Page<PermissionView>：权限码、资源、操作、系统等。

所需权限 rbac:permission:view

目标数据范围 需要 permission:view；按资源/系统筛选权限定义，未见组织作用域筛选。

失败处理 参数不合法 400；无 permission:view 403。

契约细节 需要 rbac:permission:view。可按业务系统、资源以及权限码或资源名称关键字筛选。 data 为分页结果，items 包含权限 ID、权限码、资源名称及类型、操作名称和系统名称。 权限码由资源编码与操作编码以冒号连接，授权接口使用权限 ID。page 默认 0、size 默认 50，实际每页限制为 1～500。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 790 行；按路径在对应 Controller、Service 中搜索。

37 POST /permissions

用途 为已有资源新增一个操作的权限

输入 JSON：resourceId、actionCode。

输出 新建 PermissionView；之后仍须另行授予角色。

所需权限 rbac:permission:create

目标数据范围 需要 permission:create；写入目标资源的一个操作权限。

失败处理 资源或操作不存在 404；相同资源操作重复 409；参数缺失 400。

契约细节 需要 rbac:permission:create。resourceId 和 actionCode 必填，资源和操作词汇必须已存在，否则返回 404。权限码自动生成为「资源编码:操作编码」；同资源、同操作只能有一条权限，重复时返回 409。 成功返回权限基本信息，并记录 PERMISSION_CREATE；权限定义创建后仍需单独授予角色。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 807 行；按路径在对应 Controller、Service 中搜索。

38 GET /permissions/heatmap

用途 权限使用热力图（业务系统 × 操作，成功鉴权次数）

输入 query：from、to，可省略，时间区间为 [from,to)。

输出 actions、systems、cells 三维索引数据，以及实际时间范围。

所需权限 rbac:heatmap:view

目标数据范围 需要 heatmap:view；统计时间段内 AUTH_SUCCESS 审计事件，未按部门裁剪。

失败处理 from≥to 400；无 heatmap:view 403。

契约细节 需要 rbac:heatmap:view。仅统计时间范围内 AUTH_SUCCESS 审计事件，时间区间为 [from, to)。 to 默认当前时间，from 默认所选 to 的 7 天前，from 必须早于 to，否则返回 400。 data 包含实际 from、to、横轴操作名称 actions、纵轴系统名称 systems，以及 cells。 每个 cell 为 [操作下标, 系统下标, 成功次数]，下标从 0 开始，未返回的组合可按 0 展示。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 831 行；按路径在对应 Controller、Service 中搜索。

菜单

39 GET /menus/me

用途 当前用户可见的菜单树（按「菜单资源:view」动态生成）

输入 Bearer 令牌；无业务参数。

输出 本人可见的 MenuNode[] 嵌套树。

所需权限 @self

目标数据范围 按本人对关联 MENU 资源的 view 权限过滤叶子；空目录不返回。

失败处理 未登录 401；无可见菜单返回空数组，不代表可绕过业务 API 权限。

契约细节 登录即可调用，无需额外操作权限。对关联 MENU 资源的节点按「资源编码:view」判定可见性， 未关联资源的目录仅在至少一个子节点可见时返回，并按 sortOrder、ID 组织为带 children 的树。 data 为菜单节点数组，包含路由、组件、图标和资源信息；接口访问权限仍由服务端独立校验。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 849 行；按路径在对应 Controller、Service 中搜索。

40 GET /menus

用途 完整菜单树

输入 无业务参数。

输出 完整 MenuNode[] 管理树，含空目录。

所需权限 rbac:menu:view

目标数据范围 需要 menu:view；返回完整目录，非按个人可见菜单过滤。

失败处理 无 menu:view 403。

契约细节 需要 rbac:menu:view。返回用于菜单管理的完整菜单树，包含空目录以及调用者个人动态菜单中不可见的节点。data 为按 sortOrder、ID 排列的根节点数组，各节点通过 children 嵌套子节点，不分页。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 862 行；按路径在对应 Controller、Service 中搜索。

41 POST /menus

用途 新增菜单

输入 JSON SaveMenu：parentId、resourceId、menuName、routePath、componentKey、icon、sortOrder。

输出 新 MenuNode，children 为空。

所需权限 rbac:menu:create

目标数据范围 需要 menu:create；资源关联须为 MENU 类型。

失败处理 菜单名缺失 400；父菜单/资源不存在 404；非法关联 400。

契约细节 需要 rbac:menu:create，menuName 必填。parentId 为空时创建根节点，否则上级菜单须存在。resourceId 为空表示目录，非空时必须关联已有 MENU 类型资源；sortOrder 默认 0。 成功返回新菜单节点，children 为空数组，并记录 MENU_CREATE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 872 行；按路径在对应 Controller、Service 中搜索。

42 PATCH /menus/{id}

用途 修改菜单

输入 路径 id；JSON SaveMenu；必须带 menuName，未传可选字段按整体覆盖规则清空或默认。

输出 更新 MenuNode，children 为空；查完整树须再次 GET。

所需权限 rbac:menu:edit

目标数据范围 需要 menu:edit；不可移动到自己或后代；资源只能挂 MENU 类型。

失败处理 目标/父菜单不存在 404；成环或资源类型不符 400。注意其覆盖语义与其他 PATCH 不同。

契约细节 需要 rbac:menu:edit，目标菜单须存在，menuName 必填。不能移到自身或后代节点下，关联资源必须为 MENU 类型。 本接口按 SaveMenu 整体覆盖可编辑字段：省略 parentId 会移为根节点，省略 resourceId 会变为目录，省略路由、组件或图标会清空对应值，省略 sortOrder 会设为 0。提交时应携带需要保留的字段。 成功返回更新节点（children 为空数组，需要完整树时重新查询），并记录 MENU_UPDATE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 892 行；按路径在对应 Controller、Service 中搜索。

43 DELETE /menus/{id}

用途 删除菜单（有子菜单时 409）

输入 路径菜单 id。

输出 data=null；只删菜单，资源与权限定义保留。

所需权限 rbac:menu:delete

目标数据范围 需要 menu:delete；不存在子节点才可删。

失败处理 目标不存在 404；有子菜单 409。

契约细节 需要 rbac:menu:delete。目标不存在时返回 404；存在子菜单时返回 409，须先删除子菜单。 成功仅删除菜单记录，关联的资源和权限定义保留，返回 data=null，并记录 MENU_DELETE。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 909 行；按路径在对应 Controller、Service 中搜索。

审计与统计

44 GET /audit-logs

用途 审计日志：谁、什么时候、在哪里、对谁、做了什么、结果如何、为什么

输入 query：eventType、actor、target、result、from、to、abnormal、page、size。

输出 Page<AuditLog>，按 ID 倒序。

所需权限 rbac:audit:view

目标数据范围 需要 audit:view；按时间 [from,to) 与其他条件交集查询；当前接口未按部门裁剪。

失败处理 时间/分页格式错误 400；无 audit:view 403；无记录返回空页。

契约细节 需要 rbac:audit:view。支持事件类型、操作者用户名、目标名称、结果和时间筛选，按日志 ID 倒序分页。 actor 精确匹配用户名，target 模糊匹配目标名称；时间范围为 [from, to)，不填的边界不参与筛选。 abnormal=true 仅保留登录失败及不存在、停用、删除主体发起的鉴权失败，仍与其他筛选条件取交集。 data 为审计日志分页结果，page 默认 0、size 默认 20；日志仅供查询。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 1002 行；按路径在对应 Controller、Service 中搜索。

45 GET /dashboard

用途 Dashboard 十一项指标

输入 无业务参数。

输出 Dashboard 十一项指标及生成时间。

所需权限 rbac:dashboard:view

目标数据范围 需要 dashboard:view；汇总平台总量和当日鉴权审计，不按部门裁剪。

失败处理 无 dashboard:view 403；无样本时比率与延迟为 0。

契约细节 需要 rbac:dashboard:view。返回用户、角色、权限总量，以及今日鉴权次数、成功率、失败率、P95/P99 延迟、 QPS、缓存命中率、异常访问次数和生成时间。用户总数不含已删除账号，包含停用和临时账号。 今日按服务端当天零点起算；QPS 为最近 60 秒的鉴权次数除以 60，比率为 0～1，延迟单位为毫秒。 异常访问含登录失败及无效主体鉴权，口径与审计日志 abnormal 筛选相同；无样本的比率、延迟返回 0。

源码定位 rbac-contract/src/main/resources/openapi/rbac-api.yaml 第 1026 行；按路径在对应 Controller、Service 中搜索。

第三部分 模拟业务系统接口

下列六项属于 rbac-mock-biz，用于证明权限中心判定会约束真实业务操作。它们不计入上述 45 项；具体请求与状态仍以模拟业务系统源码为准。

GET /oa/documents

用途 按部门列出发文

输入 query：deptId；Authorization 请求头。

输出 Doc[]：id、title、deptId、draftedBy、status。

所需权限 oa:m01:f02:view

目标数据范围 把 deptId 送入 /authz/check；获准后只返回该部门文档。

失败处理 缺令牌/令牌无效 401；权限中心拒绝 403。

源码定位 rbac-mock-biz/src/main/java/com/xmu/rbac/mockbiz/MockBizController.java。

POST /oa/documents

用途 在指定部门拟稿

输入 JSON：deptId、title；Authorization 请求头。

输出 新 Doc，状态 SUBMITTED，draftedBy 为当前令牌主体。

所需权限 oa:m01:f02:create

目标数据范围 以提交的 deptId 判定后创建该部门发文。

失败处理 缺令牌/令牌无效 401；无创建权限 403。

源码定位 rbac-mock-biz/src/main/java/com/xmu/rbac/mockbiz/MockBizController.java。

POST /oa/documents/{id}/approve

用途 审批指定发文

输入 路径：文档 id；Authorization 请求头。

输出 更新后的 Doc，状态 APPROVED。

所需权限 oa:m01:f02:approve

目标数据范围 先查文档，再以文档自己的 deptId 判定；普通员工若缺审批权限则拒绝。

失败处理 文档不存在 404；缺令牌 401；无审批权限 403。

源码定位 rbac-mock-biz/src/main/java/com/xmu/rbac/mockbiz/MockBizController.java。

POST /hr/attendance

用途 新建考勤记录

输入 JSON：deptId、date，可选 userId；Authorization 请求头。

输出 新 Attendance，状态 NORMAL，含 userId、deptId、date。

所需权限 hr:m04:f01:edit

目标数据范围 以 deptId 判定；selfOnly=true 时 userId 只能是令牌主体。

失败处理 缺令牌 401；无编辑权限或违反 SELF_ONLY 403。

源码定位 rbac-mock-biz/src/main/java/com/xmu/rbac/mockbiz/MockBizController.java。

PUT /hr/attendance/{id}

用途 更新考勤记录状态

输入 路径：考勤 id；JSON：status；Authorization 请求头。

输出 更新后的 Attendance；保留原 userId、deptId、date。

所需权限 hr:m04:f01:edit

目标数据范围 以原记录 deptId 判定；selfOnly=true 时仅可编辑自己的记录。

失败处理 记录不存在 404；缺令牌 401；无权限或违反 SELF_ONLY 403。

源码定位 rbac-mock-biz/src/main/java/com/xmu/rbac/mockbiz/MockBizController.java。

GET /hr/attendance

用途 按部门列出考勤

输入 query：deptId；Authorization 请求头。

输出 Attendance[]。

所需权限 hr:m04:f01:view

目标数据范围 以 deptId 判定；selfOnly=true 时进一步只保留 userId 为本人的记录。

失败处理 缺令牌 401；无查看权限 403。

源码定位 rbac-mock-biz/src/main/java/com/xmu/rbac/mockbiz/MockBizController.java。

第四部分 E 答辩 31 问

以下内容汇入上一版 E-defense-deep-QA.md，覆盖 ER 图、组织与权限导入、代码调用链及 API 高频追问。答辩时优先讲前四个 E 接口，再根据老师的问题跳到对应题目。

01 老师：ER 图一共有多少个实体？为什么不是只画用户、角色、权限三张表？

建议回答 当前实际迁移共建 14 张表：V1 有 13 张，V2 增加 sys_grant_version。用户、角色、权限只是 RBAC0 的抽象概念，落地时还要保存多对多指派、资源与操作、组织关系、菜单和审计。可分成四组记：组织与人（sys_company、sys_department、sys_user）；资源与权限（sys_software_system、sys_function_module、sys_resource、sys_action、sys_permission）；授权关系（sys_role、sys_role_permission、sys_user_role）；菜单、审计和版本（sys_menu、sys_audit_log、sys_grant_version）。菜单独立成表，因为页面导航的层级、路由与显示顺序不等同于权限定义；是否看得到叶子菜单再关联相应资源的 view 权限。这样既能回答“用户能做什么”，也能回答“对哪个部门的数据做”。

定位 数据库说明 §2（../docs/03-database.md）、V1 建表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、V2 版本表（../rbac-contract/src/main/resources/db/migration/V2__grant_version.sql）。

02 老师：为什么要单独建公司和部门？把公司当成根部门不行吗？

先澄清 V1 的 (company_id,parent_id,dept_name) 唯一键约束同级重名；根节点 parent_id 为空时，数据库唯一键本身不能可靠保证“每公司恰好一个根”，导入方法另做一次根节点检查。因此不要说这个约束完全由数据库保证。

建议回答 公司是多公司隔离和导入时识别企业的主体，保存唯一的 company_code；部门是每家公司内部的组织树，有 company_id、parent_id 和 path。把二者拆开，就能明确“这棵部门树属于哪家公司”，也允许不同公司都出现名为“财务部”的部门。公司列表 API 返回可见公司的编号和根部门编号，部门树 API 再根据公司编号查树。若只靠部门名称判断归属，同名部门很容易把员工或授权范围放错公司。

定位 V1 公司与部门表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、OrgService.companies（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/OrgService.java）、ImportService.importDepartments（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

03 老师：`parent_id`、`path`、`level` 为什么同时存在？

先澄清 若以后实现“移动部门”，必须同步更新该部门和所有后代的 path、level；当前导入逻辑主要创建或复用部门，不能宣称已实现完整迁移子树。

建议回答 parent_id 表示直接上级，便于构造树；path 例如 /3/15/27/，表示整条祖先链，判断某部门是否在授权部门的子树中可以做路径前缀比较；level 用于层级展示和排序。它们有部分信息冗余，但用途不同：查询完整子树时，path 避免每次递归；页面建树时，parent_id 更直接。创建新部门时要先保存拿到 ID，再补写完整路径。

定位 V1 部门表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、ImportService.importDepartments（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）、DepartmentMapper.findSubtree（../rbac-structured/src/main/java/com/xmu/rbac/structured/mapper/DepartmentMapper.java）。

04 老师：部门有层级，这是不是已经实现 RBAC1 的角色继承？

建议回答 不是。部门树表达组织结构与授权适用范围；RBAC1 的角色继承是“高级角色自动拥有低级角色的权限”，本阶段没有角色继承表或继承计算。这里的角色只通过 sys_role_permission 拥有哪些权限，用户经 sys_user_role 在某个部门及其子树内持有角色。不能把“部门的父子关系”说成“角色的父子关系”。

定位 数据库设计原则（../docs/03-database.md）、V1 角色、角色权限与用户角色（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）。

05 老师：一个员工只能属于一个部门，为什么还能在多个部门有权限？

建议回答 sys_user.department_id 表示员工的人事归属，只有一个；sys_user_role.scope_dept_id 表示一次角色指派在哪个部门及其下级生效，同一用户可有多条指派。因此“人在哪儿工作”和“在哪些部门能执行操作”分开建模。比如员工甲属于总公司财务部，也可以另获道路分公司上的某角色；授权范围不应靠更改其人事部门来表达。

定位 V1 用户表与用户角色指派（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、数据库说明 §3.4（../docs/03-database.md）。

06 老师：资源、操作、权限为什么要分三类？为什么不直接把“查看考勤”写成字符串？

建议回答 资源 sys_resource 是功能点，例如“考勤管理”；操作 sys_action 是可复用的动词，例如 view、edit；权限 sys_permission 是“资源 × 操作”的组合，例如 考勤管理:view。这样同一功能点能有多个操作，角色也能只获其中一部分。权限码方便接口传输与查询，但数据库仍通过资源 ID、操作 ID 和唯一键保证这个组合有明确来源，避免只凭字符串、无法关联业务资源。

定位 V1 系统、模块、资源、操作、权限表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、数据库说明 §3.3（../docs/03-database.md）。

07 老师：`sys_role_permission` 和 `sys_user_role` 为什么不是一张表？

建议回答 前者回答“这个角色具有什么权限”，键是角色与权限，并记录 self_only；后者回答“哪个用户在哪个部门范围持有这个角色”，键是用户、角色和适用部门。若合并，每给一个用户指派角色都要复制一整套角色权限，修改角色模板也难以统一生效。拆分后，导入权限清单可以更新模板的角色权限，而用户的角色指派依旧由管理员人工完成。

定位 V1 两张关联表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、ImportService.importPermissions（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

08 老师：为什么部门负责人既有 `manager_name` 又有 `manager_user_id`？负责人是否自动获得权限？

先澄清 当前 linkManagers 只从本轮新建员工中找负责人；重复导入改成已有员工时可能无法更新，已记为 HR-003 待复现问题。不要把这个边界隐瞒成“完全支持增量同步”。

建议回答 原始部门表给的是姓名，不保证在员工表中恰好唯一，所以先保留 manager_name 原始登记；若能唯一对应某个员工，再写 manager_user_id 便于关联展示。负责人只是组织资料，代码没有因关联负责人而插入 sys_user_role，所以不会自动产生管理权限。真正授权还要走人工角色指派。

定位 V1 部门表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）、ImportService.linkManagers（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）、E 审查问题（problems/E/E-review-20261026.md）。

09 老师：为什么审计日志和授权版本表没有与所有业务表画外键？

建议回答 审计记录要在用户或角色后来变化、甚至删除后仍能说明“当时谁做了什么”，所以保存操作者与目标的 ID、名称快照，不依赖业务对象继续存在。授权版本表是一个独立的技术性单行表，记录授权数据的当前版本，供缓存失效使用，不代表某个用户或角色的业务关系。二者目的分别是历史追溯和一致性控制。

定位 数据库说明 ER 图注释（../docs/03-database.md）、V2 授权版本表（../rbac-contract/src/main/resources/db/migration/V2__grant_version.sql）。

10 老师：V1 有 `expires_at`，你们 ER 图里为什么有的有、有的没有？

建议回答 V1 是历史建表迁移，原设计中 expires_at 表示账号到期时间；V4 按老师要求先停用已到期而仍启用的账号，再删除这个字段。当前实际表结构应看 V1→V4 顺序执行后的结果，不是只看 V1 文件。现存可编辑 StarUML 源文件仍有旧字段，而新 Mermaid ER 已按 V4 更新，这是已记录的模型同步问题，不能说数据库还保留旧字段。

定位 V4 删列迁移（../rbac-contract/src/main/resources/db/migration/V4__drop_user_expiry.sql）、E-04 记录（problems/E/E-review-20261026.md）。

11 老师：E 的四个 API 分别服务什么场景？为什么拆成四个？

建议回答 GET /companies 让管理界面先选可见公司，返回 CompanyView；GET /departments/tree?companyId=... 再查该公司的可见部门树，返回 DeptNode；POST /imports/organization 接收部门和员工两份表，建立组织数据，返回 ImportReport；POST /imports/permissions 接收功能权限清单，建立资源、权限和角色模板授权，也返回 ImportReport。查询与导入的读写职责不同；组织表与权限清单的数据结构、校验、权限要求和副作用也不同，因此分开。对应权限分别为 rbac:org:view、rbac:org:view、rbac:org:import、rbac:permission:create。若老师问“对应资源”，先确认是指 API 返回的业务对象，还是 RBAC 权限模型里的资源：后者的 rbac:org、rbac:permission 是受保护的管理功能资源，不是公司记录或 Excel 文件本身。

定位 OpenAPI 四接口（../rbac-contract/src/main/resources/openapi/rbac-api.yaml）、CatalogController 入口（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/CatalogController.java）。

12 老师：为什么公司列表和部门树不合成一个接口？

建议回答 公司列表只返回选择器需要的少量摘要，如公司编号、编码、名称和根部门编号；部门树可能有很多层节点，且要按操作者的管理范围过滤。先列可选公司，再按 companyId 查对应树，可避免页面一开始加载所有公司的完整树，也使“公司不存在返回 404”和“公司存在但没有可见部门返回空数组”可以分开表达。更重要的是公司是否出现、部门节点是否可见都由服务端判定，不能只靠前端隐藏选项。

定位 OrgService.companies/tree（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/OrgService.java）、CatalogDtos.CompanyView/DeptNode（../rbac-structured/src/main/java/com/xmu/rbac/structured/dto/CatalogDtos.java）。

13 老师：组织导入 API 为什么一次上传两张表？请求和响应具体是什么？

建议回答 员工行的部门归属依赖部门树，所以一个组织导入请求先处理 departments，再处理 employees，最后关联部门负责人。请求是 multipart/form-data：query 有 companyCode、companyName，文件字段名是 departments、employees。成功响应含 counts（创建、跳过等数量）、notes（处理说明）和 anomalies（表、行、原因、值），便于管理员审核，不能只显示“导入成功”。

定位 CatalogController.importOrganization（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/CatalogController.java）、ImportReport（../rbac-structured/src/main/java/com/xmu/rbac/structured/dto/ImportReport.java）、前端 orgApi（../rbac-web/src/api/index.ts）。

14 老师：为什么功能权限清单导入不是给员工直接分配角色？

建议回答 清单描述的是“某类角色模板对某功能点应有哪些操作”，所以它修改 sys_role_permission；是否让员工持有某角色属于组织授权决策，要由管理员另行写入 sys_user_role。若按岗位或负责人自动分配，原始 Excel 的错误、同名或过时岗位会直接变成越权。代码末尾的组织导入说明也明确“不分配任何角色”。

定位 ImportService.importOrganization（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）、V1 两张授权表（../rbac-contract/src/main/resources/db/migration/V1__rbac0_baseline.sql）。

15 老师：已有接口权限的人，是不是就能看到所有公司的组织树？

建议回答 不是。拦截器先检查“能否调用 org:view 这类接口”；OrgService 再用 AuthorizationService.scopeOf 取得该用户持有这项权限的部门路径，只返回范围内节点及用于显示路径的祖先。祖先可见不代表有权管理祖先的其他分支。要讲清“接口操作权限”和“目标数据范围”是两道不同的检查。

定位 OrgService.tree（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/OrgService.java）、AuthorizationService.scopeOf（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/AuthorizationService.java）、组织树测试（../rbac-structured/src/test/java/com/xmu/rbac/structured/integration/ManagementApiTest.java）。

16 老师：权限清单导入 API 为什么要返回异常清单，而不是遇到一处错就全部失败？

建议回答 原清单可能同时有大量有效行和少量无法识别的单元格。现实现对能够定位且合法的数据继续处理，把异常放进 anomalies 供人工修正；空白权限格不等于明确的“无”，为避免误撤权，会保留该资源在对应模板角色上的旧授权。若功能点连系统或模块都无法定位，本次会暂停“删除同步”，以免在不知道源清单全貌时大量误回收。严重的文件错误或必填参数缺失仍会抛异常，不能概括成“任何错误都只跳过一行”。

定位 ImportService.parseCell/importPermissions/revokeMissing（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

17 老师：后端分几层？各层谁负责、为什么这样分？

建议回答 这次是结构化实现，按功能流程拆成 Controller、Service、Mapper 三层。Controller 绑定 HTTP 参数并返回统一格式；Service 做业务校验、组织定位、事务和授权规则；Mapper 是 Spring Data JPA 接口，负责读写 MySQL，Redis 访问也放数据访问层。依赖从上向下，Controller 不直接操作 Mapper，Mapper 不做权限决策。这个项目的“Mapper”是层名，不意味着使用 MyBatis。具体到写库，Service 构造 Company、Department、User 等实体后调用 Mapper 的 save/saveAll；这些 Mapper 继承 JpaRepository，Spring Data JPA 提供基本持久化实现。复杂查询才用方法名、JPQL 或原生 SQL 明确写出。

定位 架构说明 §2.2–2.3（../docs/02-architecture.md）、CatalogController（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/CatalogController.java）、DepartmentMapper（../rbac-structured/src/main/java/com/xmu/rbac/structured/mapper/DepartmentMapper.java）。

18 老师：从网页点“导入组织与员工”后，调用链具体怎么走？要说到数据库。

建议回答 OrgView.vue 收集公司编码、名称和两个文件；orgApi.importOrganization 组装 FormData 发 POST；PermissionInterceptor 按契约检查身份及 rbac:org:import；CatalogController.importOrganization 取参数与文件流，调用 ImportService.importOrganization；Service 查找或新建公司、读工作簿、建部门树、定位员工、关联负责人；

CompanyMapper、DepartmentMapper、UserMapper 把实体写到 sys_company、sys_department、sys_user。返回 ImportReport 后前端展示计数与异常并刷新公司及部门树。顺着这条链说，比只报类名更有说服力。

定位 OrgView.vue（../rbac-web/src/views/OrgView.vue）、前端 orgApi（../rbac-web/src/api/index.ts）、Controller（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/CatalogController.java）、ImportService（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

19 老师：`importDepartments` 为什么有 `pending`、`done` 和多轮循环？

建议回答 Excel 的行顺序不保证父部门一定排在孩子前。方法把未处理部门放 pending，每轮只处理上级已经在 done 的行，直到没有待处理行或一轮没有进展。若最后仍有待处理行，通常是上级缺失或形成环，列入异常而不猜一个上级。创建部门时先保存获取自增 ID，再补写 path。这是按依赖拓扑逐步建树，不是简单按 Excel 行号插库。

定位 ImportService.importDepartments（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

20 老师：`locate` 函数做什么？为什么不直接按部门名称查？

建议回答 locate(root, child, branch, deptName) 用员工表的“分公司 + 部门”两列定位部门。若分公司是总公司，就在根节点的直接下级找；若“部门”本身就是分公司，该员工归分公司直属；否则在指定分公司的下级找组合名或原部门名。这样“财务部”不会因为重名而落到总公司或另一个分公司的财务部。无法唯一定位时记异常，不随便取第一条。

定位 ImportService.locate（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

21 老师：`readSheet`、`parseCell`、`revokeMissing` 各自负责什么？

先澄清 组织 readSheet 丢掉物理行号，下游 i+2 可能报错行，已记为 HR-002 待复现。

建议回答 readSheet 读取组织工作簿首个工作表，寻找表头、把数据行转换为文本并跳过全空行；parseCell 把权限清单单元格中的“查看/编辑本人”“全部”“无”等转换为操作集合和 selfOnly，空白或未知词返回异常；revokeMissing 比较原有业务授权与本次清单要求的 wanted，但跳过 untouched 保护的异常资源，然后回收不再要求的授权。讲函数时先说输入、输出，再说异常和数据库副作用。

定位 ImportService.java（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）、E 审查问题（problems/E/E-review-20261026.md）。

22 老师：重新导入权限清单时，为什么有 `wanted`、`untouched`、`revokeMissing`？

建议回答 wanted 是新清单明确要求的权限 ID 与“只限本人”标记；untouched 是该角色列解析异常、不应动其旧授权的资源 ID；revokeMissing 执行最后的差集回收：旧授权既不在 wanted、所属资源也不在 untouched，才删除。举例：原来能查看考勤，新表写“查看”继续保留；写“无”表示明确回收；留空报异常、暂时保留。若整行找不到系统/模块，本次跳过所有删除同步。这里删除的是角色模板的业务授权，不碰权限中心自身的 rbac:* 管理权限。

定位 ImportService.importPermissions/revokeMissing（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）、

RolePermissionMapper.findBusinessGrantsOfRole（../rbac-structured/src/main/java/com/xmu/rbac/structured/mapper/RolePermissionMapper.java）。

23 老师：为什么权限更新后还要调用 `bumpGrantVersion`？Redis 缓存不能直接删吗？

建议回答 授权数据改在 MySQL，旧授权可能已缓存在 Redis。如果只逐个删缓存键，数据库提交与删键之间、以及并发回填时容易留下短暂不一致。项目把全局授权版本号放在 MySQL 的单行表里，修改授权时在同一事务中递增；新请求先读版本号，用新版缓存键，查不到就从数据库加载。因此版本变化随事务一起提交或回滚。这里是代码设计解释；缓存下一次请求效果是否完全符合要求，仍应以集成实测为证。

定位 V2 迁移（../rbac-contract/src/main/resources/db/migration/V2__grant_version.sql）、AuthorizationService.bumpGrantVersion（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/AuthorizationService.java）。

24 老师：`@Transactional` 是什么意思？导入有异常时到底回滚还是继续？

建议回答 ImportService.importOrganization 和 importPermissions 的业务写入各在一个事务中。代码主动收集并跳过的逐行/逐格异常不会抛出事务异常，所以有效数据可继续写入并最终提交；无法读取文件、必填参数缺失或数据库异常等被抛出的错误会使业务事务回滚。这里要区分“报告里有 anomalies”和“Java 方法抛异常”：前者不等于整笔失败。审计的 AuditService.record 用独立事务，审计记录与业务回滚的关系需单独核对，不能简单说所有表绝对一起回滚。

定位 ImportService 两个导入入口（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）、AuditService.record（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/AuditService.java）。

25 老师：现有实现有哪些明确边界？你作为 Debug 负责人发现了什么？

建议回答 我们在代码静态审查中记录了三类 E 风险：组织导入跳过标题/空白行后异常报告的原始行号可能偏移（HR-002）；重复导入时负责人查找只看本轮新建员工，可能使新负责人姓名与旧用户 ID 不一致（HR-003）；同模块同名但不同义的功能点换序，依赖出现次序的资源认领可能错配授权（HR-004）。另外 V4 已删 expires_at，但可编辑 StarUML 旧模型未同步（E-04）；组织导入页面复开弹窗可能保留旧报告（E-05）；端到端测试有弹窗外文案的断言范围问题（E-06）。这些是有代码路径证据的待复现或文档问题，不能声称已全部实测、修复。

定位 E 问题证据（problems/E/E-review-20261026.md）、E 阅读状态（E.md）。

26 老师：45 个接口是不是太多？接口越多越容易出问题，你们为什么这样拆？

建议回答 数量本身不能证明设计合理。当前契约按认证、鉴权、用户、角色、权限、菜单、组织、统计八组列出 45 个操作，分别对应不同的查询、写入、授权和管理场景。以 E 为例，公司列表、按公司查部门树、组织导入、权限清单导入，输入结构、授权要求及副作用不同，合成一个“万能接口”会让权限和错误语义更模糊。反过来说，每多一个接口就多一处鉴权、参数、状态码与测试的维护面。因此我们的答辩重点不是“多就是好”，而是每个接口能说清场景、请求/响应、所需权限、数据范围、错误边界，并由契约与实现双向核对。不能把“45 个都在契约里”当作没有漏洞的证明。

定位 接口总表（../docs/04-api.md）、OpenAPI 契约（../rbac-contract/src/main/resources/openapi/rbac-api.yaml）、启动核对（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/WebConfig.java）。

27 老师：你们为什么全用 PUT，不用 PATCH？

先澄清 这句是别组的追问，不符合本仓库。当前权限中心的 45 个操作按契约统计是 GET 19、POST 17、PATCH 4、DELETE 5、PUT 0。四个 PATCH 是 PATCH /users/{id}、PATCH /users/{id}/status、PATCH /roles/{id}、PATCH /menus/{id}。rbac-mock-biz 示例业务接口另有 PUT /hr/attendance/{id}，不能算进权限中心的 45 个接口。

建议回答 一般来说，PUT 更适合客户端提交某资源的完整目标表示，PATCH 适合只修改部分字段；HTTP 方法选择还要与实际请求体语义一致。我们的用户和角色更新是典型部分更新：未提交的字段保持旧值。菜单虽然路由名为 PATCH，但契约明确规定 SaveMenu 整体覆盖可编辑字段；遗漏 parentId、resourceId、路由等会改变或清空旧值。这个特例不是理想的部分更新语义，答辩时应如实说明，不能笼统声称“四个 PATCH 都只改提交的字段”。可改进为真正的部分更新，或重新评估用 PUT 并在契约中要求完整可编辑表示；改法需同时改前端和测试。

定位 方法总表（../docs/04-api.md）、菜单 PATCH 契约（../rbac-contract/src/main/resources/openapi/rbac-api.yaml）、MenuService.apply（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/MenuService.java）、用户更新（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/UserManagementService.java）、角色更新（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/RoleManagementService.java）。

28 老师：能给出一个 PATCH 与 PUT 的具体例子吗？少传字段会怎样？

建议回答 用户原资料是 {realName: "张三", phone: "123", position: "职员"}。调用 PATCH /users/{id} 只传 {phone: "456"}，UserManagementService.update 只对非 null 字段赋值，因此姓名和岗位保持不变。它还能用空字符串清空电话或岗位，这是“未提交”和“提交为空”的区别。角色的 PATCH /roles/{id} 也按非 null 字段更新。菜单是反例：PATCH /menus/{id} 只传 {menuName: "新名称"} 会把父菜单移到根、解除资源关联并清掉一些展示字段；因为契约就规定了覆盖，所以前端需要提交所有要保留的字段。若老师说这更像 PUT，应承认接口命名与实质语义存在可讨论处。

定位 UserManagementService.update（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/UserManagementService.java）、RoleManagementService.update（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/RoleManagementService.java）、SaveMenu 契约（../rbac-contract/src/main/resources/openapi/rbac-api.yaml）。

29 老师：为什么导入用 POST，查询用 GET？反复上传同一文件是否保证结果完全不变？

建议回答 GET /companies 和 GET /departments/tree 只读，适合 GET；两种导入接收 Excel 并创建或调整数据库记录、审计和授权版本，适合 POST。组织导入对已经存在的用户名会跳过，权限导入会对角色模板做增删同步，但这不等于整个 POST 被严格保证为幂等操作：文件变化、数据库现状、异常行、审计等都可能改变结果。不能回答“重复调用一定完全一样”，应看导入报告的新增、跳过、异常与授权变化。

定位 CatalogController（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/CatalogController.java）、ImportService（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

30 老师：仅凭 `x-required-permission`，接口就安全吗？如果手工改 URL 里的 ID 呢？

建议回答 入口权限只判定“能不能调用这类操作”。随后 Service 还要检查目标是否存在、是否属于操作者可管理的公司/部门，以及业务限制。例如组织树按 scopeOf 裁剪；用户调部门会检查目标公司相同且新部门也在编辑范围内。老师若手工把 companyId 或用户 ID 改成别人的，不能靠前端隐藏按钮防护。反过来，如果某个 Service 忘记目标范围检查，只有契约权限是不够的，这也是接口审查重点。

定位 PermissionInterceptor（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/PermissionInterceptor.java）、OrgService（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/OrgService.java）、UserManagementService.update（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/UserManagementService.java）。

31 老师：为什么说 API 是容易被追问的地方？你们应该怎么证明它？

建议回答 老师可以从契约任选一条路由，连续问“谁用、谁能调、请求体有哪些字段、为什么选这个方法、返回什么、错了是什么状态、Service 改哪些表、目标范围谁校验”。准备时按一条具体请求从前端到数据库走通，比背接口数量有效。以 POST /imports/organization 为例，要能说明两份文件、公司参数、rbac:org:import、先部门后员工、ImportReport 中的异常、事务行为，以及导入不会自动给员工分配角色。现有自动测试和契约核对是证据，但不能据此声称所有边界都实测过。

定位 E 四接口说明（../docs/04-api.md）、CatalogController（../rbac-structured/src/main/java/com/xmu/rbac/structured/controller/CatalogController.java）、ImportService（../rbac-structured/src/main/java/com/xmu/rbac/structured/service/ImportService.java）。

材料来源与核对边界

rbac-contract/src/main/resources/openapi/rbac-api.yaml：45 项权限中心接口与权限声明。

docs/04-api.md：通用响应、错误码、关键业务语义及模拟业务系统接口。

rbac-structured/src/main/java/com/xmu/rbac/structured/controller 与 service：目标对象范围、事务和业务错误。

human-review/E-defense-deep-QA.md：E 的 31 个答辩追问。

Word 内容为验收准备材料，不替代合同；若老师现场提问“是否实测”，应提供测试运行记录，不能只展示本文档。